MOBILE APPLICATION AND DEVELOPMENT • WEEK 4 • THEORY 2

ConstraintLayout & XML Layout Files

Pin it, stretch it, balance it · and how XML becomes a real screen

Ekramul Haque Amin · Lecturer, AI & Data Science · Dhaka International University

TODAY'S ROADMAP

What We Will Cover

1. Why nested layouts hurt, and how ConstraintLayout fixes it
2. Constraints: pinning, centring, stretching, bias, chains and guidelines
3. Building a login screen the ConstraintLayout way
4. Inside an XML layout file: namespaces, ids and resources
5. How XML turns into a screen: inflation, findViewById and ViewBinding

LAST CLASS

Recap: Two Layouts So Far

LinearLayout
One row or one column. Weights share leftover space.

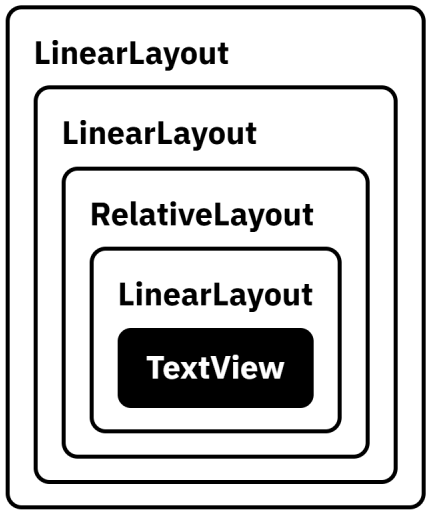
RelativeLayout
Place views relative to the parent or a neighbour.

The catch
Complex screens end up nested inside nested layouts.

Today’s question: how do we build complex screens without nesting everything?

THE NESTING PROBLEM

The Pyramid of Layouts



- Every extra level is more measuring and positioning work for Android
- Deep XML is hard to read and harder to change
- One small design change can mean rewiring several levels

PART 1 — CONSTRAINTLAYOUT

Think of Rubber Bands

A ConstraintLayout holds every view with invisible rubber bands. Each band ties one side of a view to the parent or to another view. All views live side by side in one flat layer, with no nesting.

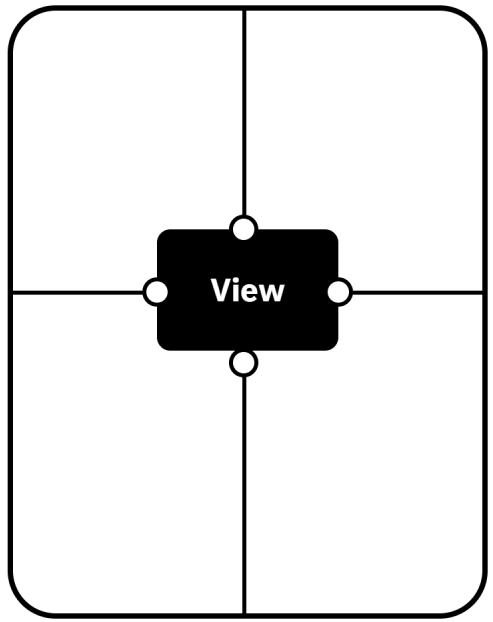
Flat
One layout holds everything, so the hierarchy stays shallow

Flexible
Centre, stretch, align and balance views with simple rules

Editor-friendly
Drag and drop in the Design tab, then read the XML it writes

LIKE PINNING A POSTER

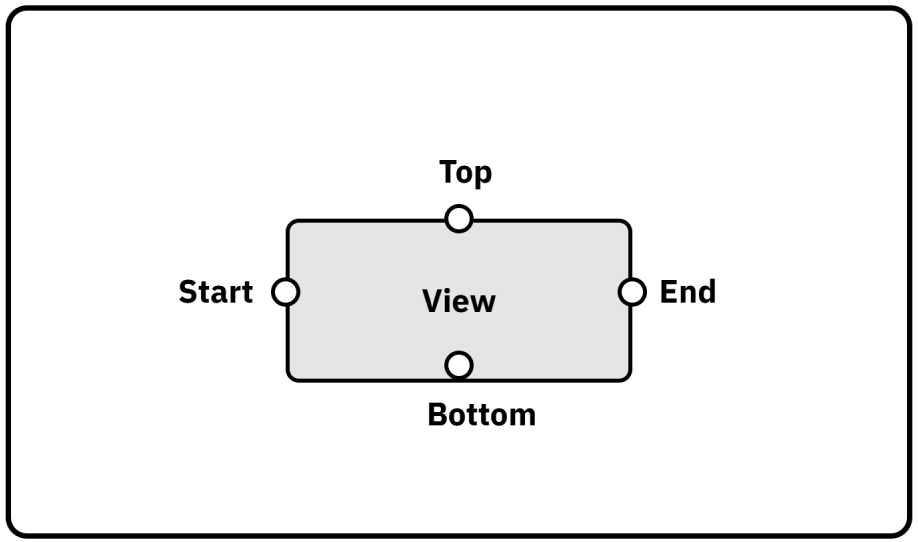
Four Pins Hold a View in Place



- Top pin: tied to the top of the parent
- Bottom pin: tied to the bottom of the parent
- Start pin: tied to the start (left) edge
- End pin: tied to the end (right) edge

EACH VIEW HAS FOUR HANDLES

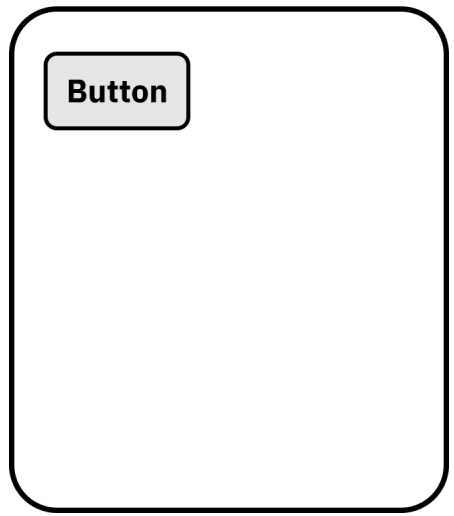
The Four Anchor Points



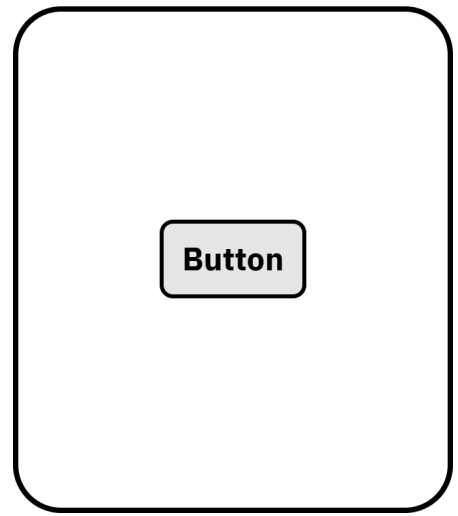
- Use Start and End, not Left and Right, so layouts flip correctly for right-to-left languages
- Each anchor can connect to the parent, a sibling’s edge, or a guideline
- The little circles are exactly what you see in the Design tab

THE ONE RULE TO REMEMBER

Every View Needs Two Constraints



No constraints: the view jumps to the top-left corner and shows a red error in the editor



One horizontal and one vertical constraint: the view stays exactly where you pinned it

At least one horizontal (start or end) and one vertical (top or bottom) constraint.

THE VOCABULARY

Reading a Constraint Attribute

Attribute	Meaning	Typical value
`layout_constraintTop_toTopOf`	My top edge ↔ their top edge	parent
`layout_constraintTop_toBottomOf`	My top edge ↔ their bottom edge	@id/txtTitle
`layout_constraintBottom_toBottomOf`	My bottom ↔ their bottom	parent
`layout_constraintStart_toStartOf`	My start ↔ their start	parent
`layout_constraintEnd_toEndOf`	My end ↔ their end	parent
`layout_constraintStart_toEndOf`	My start ↔ their end	@id/imgAvatar

Pattern: layout_constraint + MySide + _to + TheirSide + Of. All of these start with app: in XML.

YOUR FIRST CONSTRAINTS

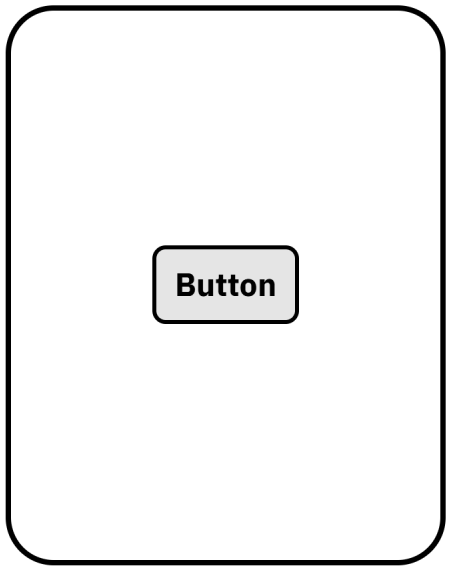
Pin a View to the Top-Left

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Two constraints: one vertical (top) and one horizontal (start). That is enough.

A HANDY TRICK

Centre a View with Four Constraints

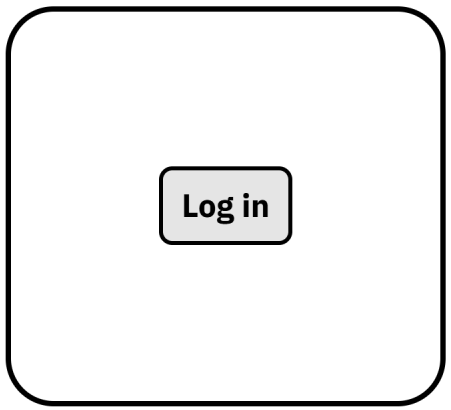



```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

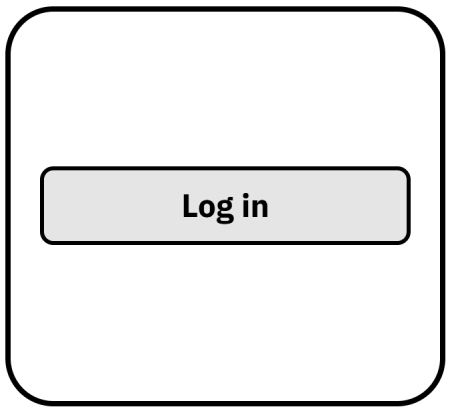
Pulled equally from both sides, the view settles in the middle.

STRETCH BETWEEN TWO PINS

0dp Means Match Constraints



layout_width="wrap_content": as wide as the text



layout_width="0dp": stretch between the start and end pins

```
android:layout_width="0dp"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintEnd_toEndOf="parent"
```

Avoid match_parent inside a ConstraintLayout. Use 0dp with constraints.

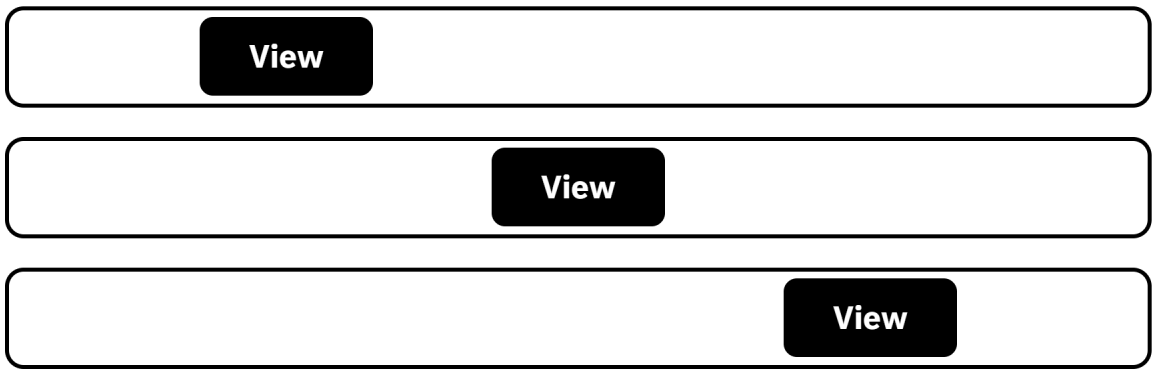
NOT EXACTLY IN THE MIDDLE

Bias: Slide a View Between Its Pins

`horizontal_bias = 0.2`

`horizontal_bias = 0.5`
`(default)`

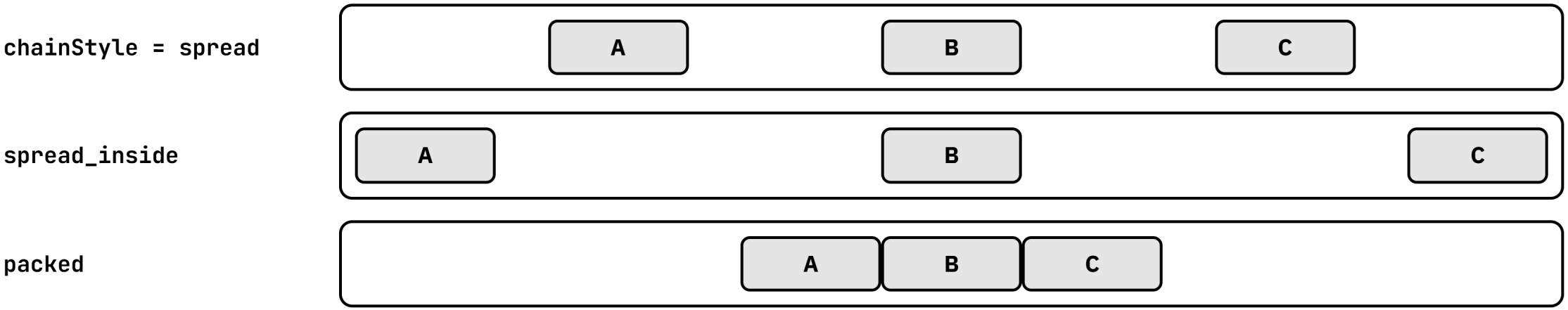
`horizontal_bias = 0.8`



Constrain start and end to the parent, then app:layout_constraintHorizontal_bias moves the view along the line.

VIEWS THAT HOLD HANDS

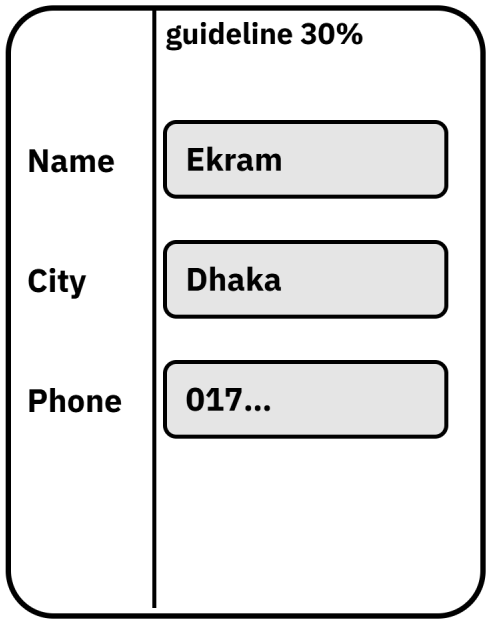
Chains



A chain forms when views are constrained to each other in both directions. The first view’s attribute sets the style.

AN INVISIBLE RULER

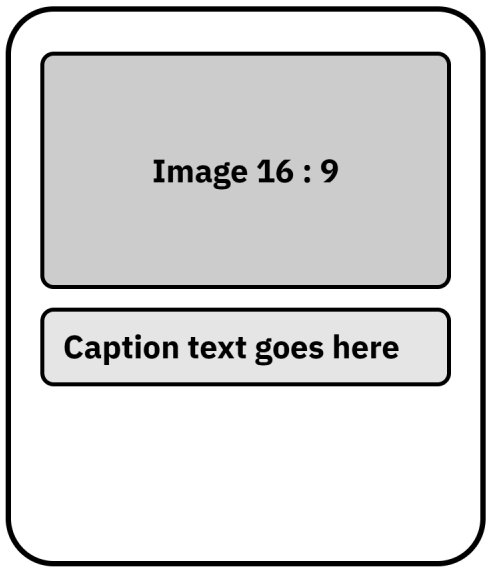
Guidelines



- A guideline is a line that never shows in the app. It exists only to align views
- Place it at a percentage, such as 30% of the width, so it adapts to every screen
- Constrain labels to its left and fields to its right and everything lines up

KEEP THE SHAPE

Aspect Ratio

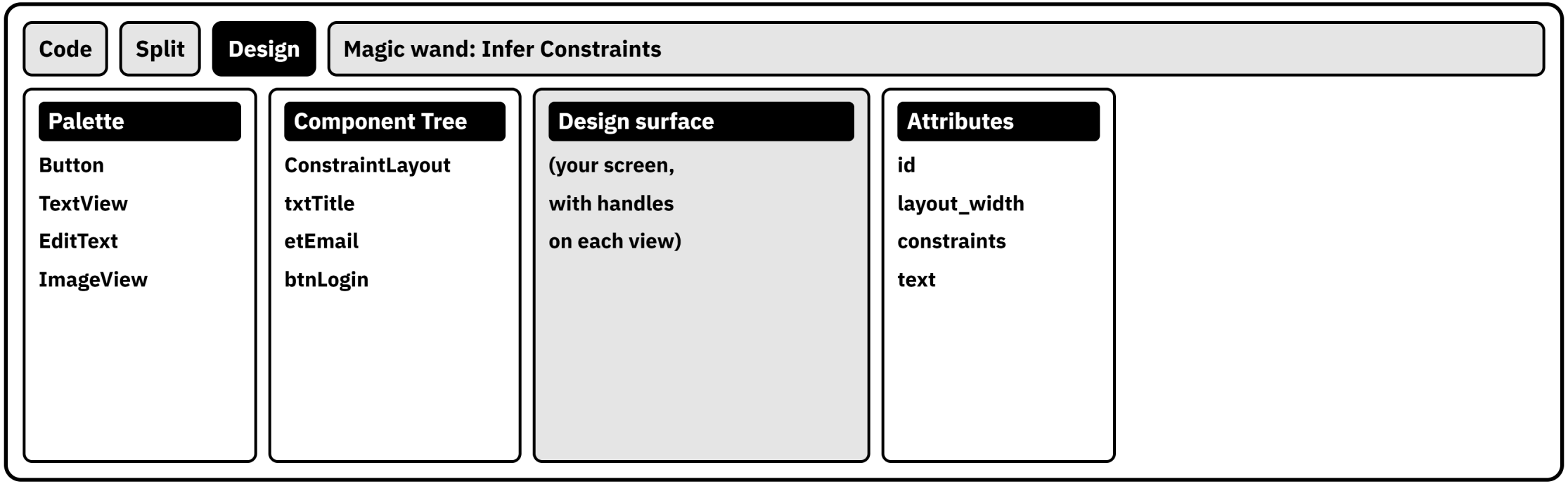



```
<ImageView
    android:layout_width="0dp"
    android:layout_height="0dp"
    app:layout_constraintDimensionRatio="16:9"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

The width stretches, and the height follows the 16:9 ratio.

ANDROID STUDIO'S LAYOUT EDITOR

A Tour of the Design Tab



Drag from the Palette to the surface, pin with handles, then fine-tune in Attributes. Switch to Code to see the XML.

LET THE EDITOR HELP

Three Handy Buttons

Autoconnect
Connects a view to the nearest edges automatically as you drop it.

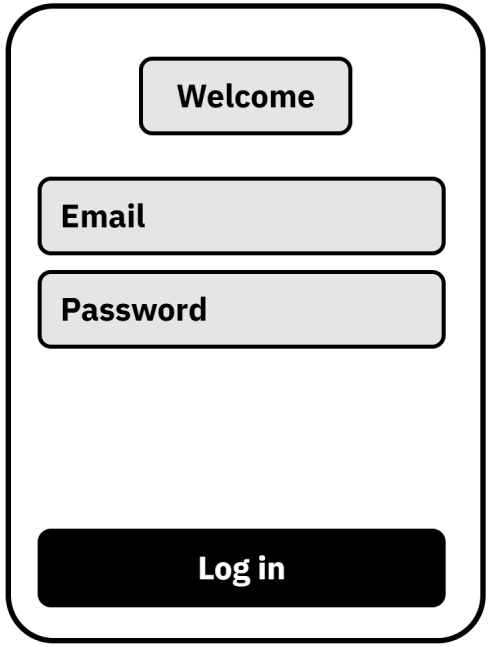
Infer Constraints
Guesses constraints from where the views sit. Check the result, because it cannot read your mind.

Clear All Constraints
Removes every constraint so you can start again.

Tip: drop views roughly where you want them, press Infer Constraints, then fix anything odd in the Attributes panel.

OUR LOGIN SCREEN, AGAIN

Login Screen with ConstraintLayout



- Title: pinned to the top, centred horizontally
- Email: below the title, start and end pinned, width 0dp
- Password: below Email, same start and end pins
- Button: pinned to the bottom of the parent

THE XML, PART 1

Title and Email

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="24dp">
    <TextView android:id="@+id/txtTitle" android:text="Welcome"
        android:layout_width="wrap_content" android:layout_height="wrap_content"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />
    <EditText android:id="@+id/etEmail" android:hint="Email"
        android:layout_width="0dp" android:layout_height="wrap_content"
        app:layout_constraintTop_toBottomOf="@id/txtTitle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />
```

THE XML, PART 2

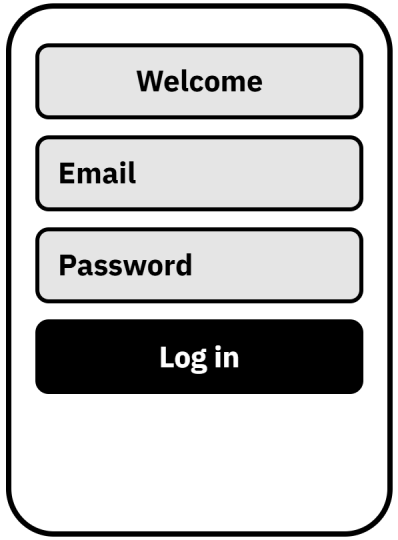
Password and Button

```
    <EditText android:id="@+id/etPassword" android:hint="Password"
        android:inputType="textPassword"
        android:layout_width="0dp" android:layout_height="wrap_content"
        app:layout_constraintTop_toBottomOf="@id/etEmail"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />
    <Button android:id="@+id/btnLogin" android:text="Log in"
        android:layout_width="0dp" android:layout_height="wrap_content"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

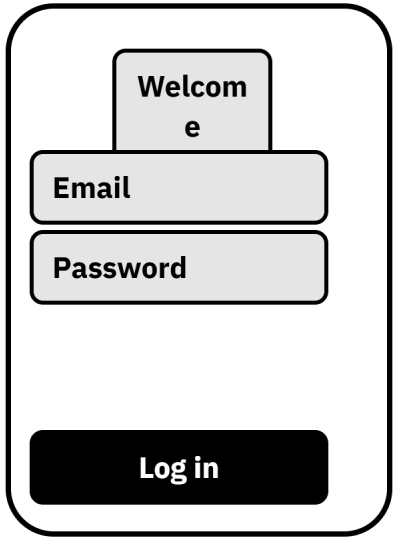
Every view has a vertical and a horizontal constraint. Use @string resources for the text in real apps.

SAME SCREEN, THREE TOOLS

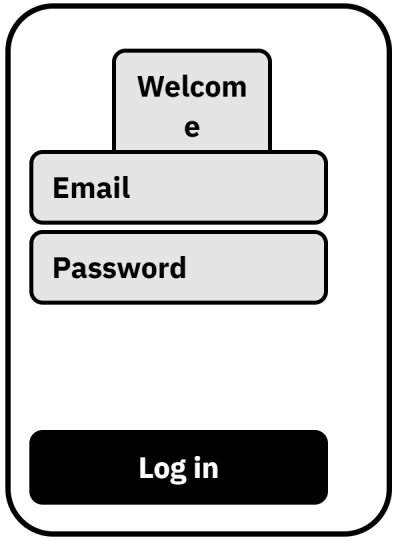
Linear vs. Relative vs. Constraint



LinearLayout: stacked in order



RelativeLayout: layout_below and alignParentBottom



ConstraintLayout: constraints to the parent and between views

All three can build this screen. ConstraintLayout stays flat and keeps working as the screen grows.

WHY DESIGNERS PREFER IT

Flat vs. Nested

```
LinearLayout
  LinearLayout
    TextView
    LinearLayout
      EditText
      EditText
  Button
```

Nested: 4 levels deep

```
ConstraintLayout
  TextView
  EditText
  EditText
  Button
```

Flat: 1 level

A flatter tree is quicker to draw, quicker to change and easier to read.

PART 2 — XML LAYOUT FILES

Where Layout Files Live

```
app/src/main/res/
    layout/
        activity_main.xml ← default screen
        item_contact.xml ← one row of a list
    layout-land/
        activity_main.xml ← used in landscape
    values/
        strings.xml colors.xml dimens.xml
    drawable/
        ic_logo.xml
```

Every screen, row and dialog is described by one XML file inside res/layout.

THE THREE PREFIXES

XML Namespaces

Prefix	What it covers	Example
android:	Built-in Android attributes	android:text
app:	Library attributes (ConstraintLayout, Material)	app:layout_constraintTop_toTopOf
tools:	Design-time only, never in the app	tools:text

```
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
```

Declare the prefixes once, on the root element.

NAMING YOUR VIEWS

@+id and @id

@+id/btnLogin
Creates a new id. Use it once, where the view is defined.

@id/btnLogin
Refers to an id that already exists. Use it in constraints and rules.

```
android:id="@+id/btnLogin" // defines it
app:layout_constraintTop_toBottomOf="@id/etPassword" // uses it
```

Pick one naming style and keep to it: txtTitle, etEmail, btnLogin.

NEVER HARD-CODE VALUES

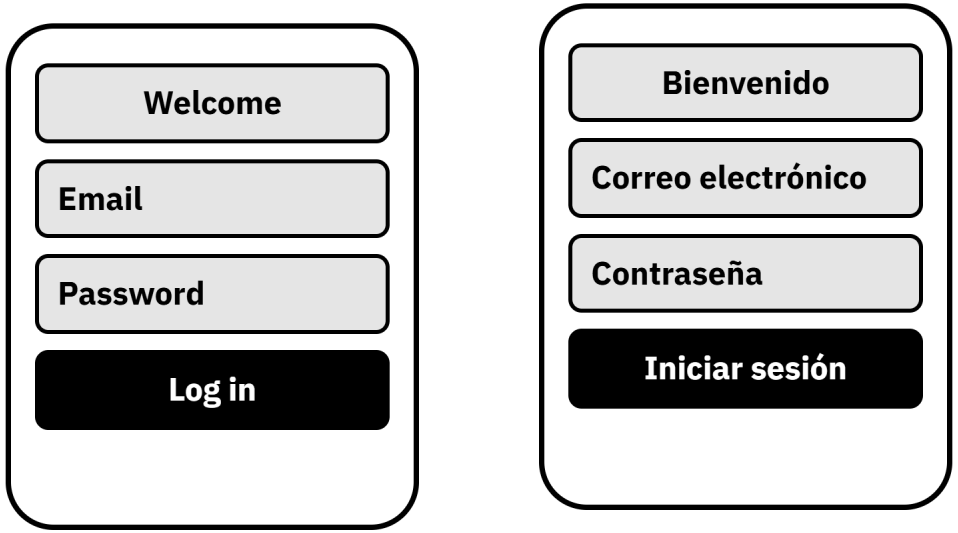
Resource References

Reference	What it points to	Lives in
@string/welcome	Text shown on screen	res/values/strings.xml
@color/brand_blue	A colour	res/values/colors.xml
@dimen/margin_normal	A size such as 16dp	res/values/dimens.xml
@drawable/ic_logo	An image or vector	res/drawable/

Change a value in one file and every screen that uses it updates.

ONE LAYOUT, MANY LANGUAGES

Strings Make Translation Easy



values/strings.xml (English)

values-es/strings.xml (Spanish)

The layout XML never changes. Android picks the strings file that matches the phone’s language (values-bn for Bangla).

A PREVIEW-ONLY TRICK

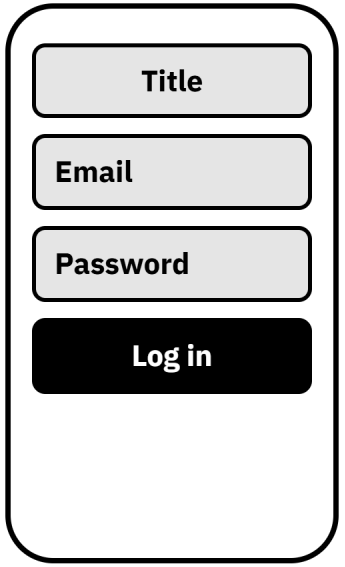
Sample Text with tools:

```
<TextView
    android:id="@+id/txtName"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/name_placeholder"
    tools:text="Ekramul Haque Amin" />
```

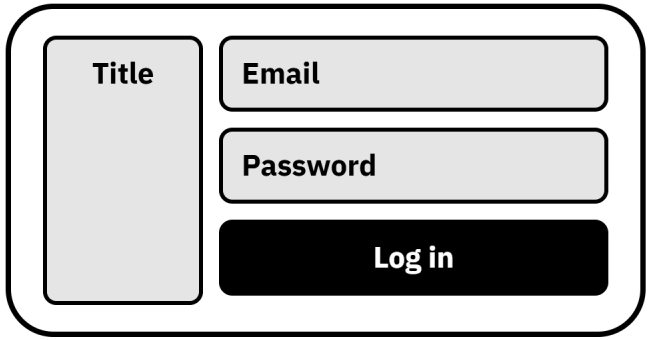
tools:text shows realistic sample text in the Design tab, so you can judge the layout. It is removed from the running app.

ROTATE THE PHONE

Portrait and Landscape Layouts



res/layout/activity_main.xml

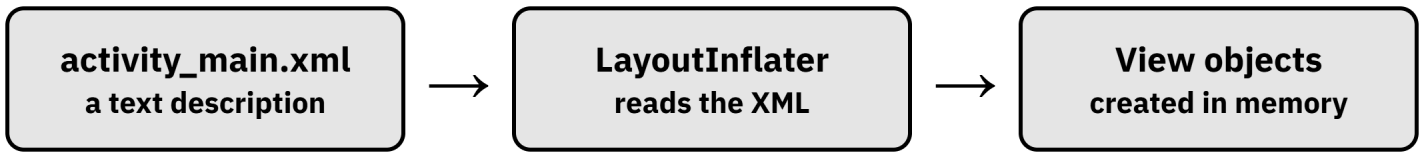


res/layout-land/activity_main.xml

Same file name, different folder. Android picks the right one automatically when the phone rotates.

FROM TEXT FILE TO TOUCHABLE SCREEN

How XML Becomes a Screen




```
public class MainActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main); // inflate the XML
    }
}
```

This is called inflation. Every tag in the XML becomes a real Java object you can change.

GRAB A VIEW FROM JAVA

findViewById()

```
Button btnLogin = findViewById(R.id.btnLogin);
EditText etEmail = findViewById(R.id.etEmail);

btnLogin.setOnClickListener(v → {
    String email = etEmail.getText().toString();
    Toast.makeText(this, "Hello " + email, Toast.LENGTH_SHORT).show();
});
```

Call it after setContentView(). R.id.btnLogin is the id you wrote in the XML.

THE MODERN WAY

ViewBinding

```
// build.gradle (module)
android {
    buildFeatures { viewBinding true }
}

// MainActivity
private ActivityMainBinding binding;
binding = ActivityMainBinding.inflate(getLayoutInflater());
setContentView(binding.getRoot());
binding.btnLogin.setOnClickListener(v → { /* ... */ });
```

activity_main.xml becomes ActivityMainBinding. No casting, no findViewById, and a typo becomes a compile error instead of a crash.

THE BRIDGE BETWEEN XML AND JAVA

What Is the R Class?

R.layout.activity_main
A layout file

R.id.btnLogin
A view id

R.string.welcome
A string resource

Android Studio generates R automatically from your res folder. You never edit it by hand.

WHICH ONE SHOULD I USE?

Choosing a Layout

Situation	Best choice	Why
A simple form or row of buttons	LinearLayout	Easy to read
Text on top of an image	FrameLayout	Views stack on each other
A few views aligned to each other	RelativeLayout	Rules between views
A complex or resizable screen	ConstraintLayout	Flat and flexible

KEEP THIS ONE

Attribute Cheat Sheet

Layout	Attributes to remember
LinearLayout	orientation · layout_weight · gravity
RelativeLayout	layout_below · layout_toEndOf · layout_alignParentBottom
ConstraintLayout	layout_constraintTop_toTopOf · 0dp · bias · chainStyle
FrameLayout	layout_gravity · children stack

LEARN FROM THESE NOW

Common ConstraintLayout Mistakes

Missing a constraint
No vertical or horizontal pin sends the view to the top-left

Using match_parent
Use 0dp together with constraints to stretch

Forgetting app:
Constraint attributes use the app: prefix, not android:

Typo in an id
@id/etEmal does not exist, so the constraint is ignored

BEFORE YOU LEAVE TODAY

Recap Checklist

1. Can you explain why ConstraintLayout keeps the hierarchy flat?
2. What two kinds of constraint does every view need?
3. When do you use 0dp instead of wrap_content or match_parent?
4. What is the difference between @+id and @id?
5. What is the difference between findViewById and ViewBinding?

WHERE WE GO FROM HERE

Next Steps

This week: Lab
Implement all three layout types: build the same screen with Linear, Relative and ConstraintLayout

Week 5
Basic UI components: TextView, EditText, Button and ImageView, then the Greetings app

Before the lab, keep your login screen project open. We will rebuild it three ways.

Questions?

Constraints | Bias & chains | XML resources | ViewBinding

Ekramul Haque Amin

Lecturer, AI & Data Science · Mobile Application and Development (0613-409)